Spring 2026 Analytics II Team Project Final Report

Group 1

Joshua Boyles
Dave Julianto
Hayden Burke
Daniel Veiner
Sebastian Hajdu

Professor:

Dr. Kaixun Hua

Date:

May 1, 2026

Abstract

Manufacturing defect detection is critical for maintaining product quality, reliability, and safety in industrial systems. In manufacturing, undetected defective components can lead to equipment failures, increased maintenance costs, scrap, rework, and potential safety risks. This project applies machine learning techniques to predict defective parts using historical manufacturing test data.

The dataset contains 229,999 observations, including 850 duplicate records, 0 missing values, and 30 numerical predictor variables representing sensor readings or process measurements. The target variable is a binary defect label, where Class 1 represents defective parts. Only 458 samples are defective, giving a defect rate of 0.1991%. This creates a severe imbalanced classification problem, where traditional metrics such as accuracy can be misleading because a model may achieve high accuracy while failing to identify rare defects.

To address this challenge, this project first compares several machine learning models. Because the defective class is extremely rare, each model is tested with different imbalance-handling strategies, including oversampling, undersampling, SMOTE, and class-weighted learning. Different data splits are also compared to check whether performance remains stable. Finally, models are ranked using a custom FN & FP Score based on false negative and false positive rates, since overall accuracy can appear high even when rare defects are missed.

1 Introduction

Manufacturing defect detection plays an important role in improving product quality, reducing waste, and preventing costly failures. In manufacturing, defective components can contribute to equipment damage, increased maintenance, production delays, customer dissatisfaction, and potential safety risks if they are not detected early. As manufacturing systems collect more process and test data, machine learning provides an opportunity to identify defect patterns that may not be obvious through manual inspection alone.

This project uses historical manufacturing data to predict whether a manufactured part is defective. Of the full dataset, only 0.1991% of samples are labeled as defective. This makes the problem a severe imbalanced classification task.

The main challenge is that the defective class is extremely rare. This project first compares several machine learning models on this unbalanced data. Because the defective class is extremely rare, each model is tested with different imbalance-handling strategies. Different data splits are also compared to check whether performance remains stable. Finally, models are ranked using a custom FN & FP Score based on false negative and false positive rates, since overall accuracy can appear high even when rare defects are missed.

The objectives of this project are to:

1. Develop a predictive model for classifying defective parts.
2. Evaluate model performance under severe class imbalance using the custom FN & FP Score.
3. Identify key process variables associated with defective production.
4. Provide insights that can support manufacturing quality control and process improvement.

Together, these objectives outline a framework for using historical manufacturing data to improve defect detection.

2 Related Work

Machine learning has increasingly been applied to manufacturing systems for predictive quality monitoring and defect detection. Prior research shows that industrial process and test data can be used to learn relationships between operating conditions and product quality outcomes.

Zahrani et al. apply machine learning models to predict product quality and groove geometry in a circular laser-grooving manufacturing process. Their results show that data-driven models can capture nonlinear relationships between process parameters and final part characteristics, supporting predictive monitoring rather than purely reactive inspection [1].

More recent studies extend this direction using stronger ensemble methods for rare-defect prediction in industrial environments. Geng et al. propose a framework for predicting internal cracks in continuous steel casting using CatBoost combined with synthetic data generation, demonstrating improved detection of infrequent defect patterns in a real manufacturing context [2].

A central challenge in manufacturing defect prediction is severe class imbalance. Defective products may represent only a small fraction of observations, causing standard training and evaluation to favor the majority class and report high accuracy while missing rare failures. Kafunah et al. examine this issue in fault detection and evaluate imbalance-aware strategies such as oversampling and reweighting to improve minority-class detection, highlighting the importance of focusing on defect-class metrics such as precision, recall, and F1 rather than accuracy alone [3].

Cost-sensitive learning provides a complementary approach by explicitly assigning higher penalties to misclassifying defective samples, reflecting the higher operational risk of false negatives in manufacturing. Liu et al. introduce a cost-sensitive defect detection framework that modifies the training objective using an explicit cost structure to prioritize costly minority-class errors, and they report improved defect identification under imbalance [4].

Building on these findings, our project systematically compares interpretable and ensemble models, including logistic regression, decision trees, random forests, and boosting under multiple imbalance-mitigation strategies. We emphasize cost-sensitive random forests and minority-class performance metrics to evaluate model effectiveness for defect detection in a highly imbalanced manufacturing dataset.

3 Model and Method

3.1 Class Imbalance Strategies

The dataset comprises 229,999 test records with only 458 defective samples (0.2%). To address the imbalance, we evaluate four dataset manipulation techniques: Random Oversampling (duplicating minority samples), Random Undersampling (removing majority samples), SMOTE (synthetic sample generation), and Class-Weighted training (adjusting learning objectives to penalize minority-class errors). Each data sampling method was tested on every considered model type.

3.2 Considered Model Types

Logistic Regression

Logistic Regression provides high interpretability, modeling defect probability as a smooth function of sensor readings so engineers can quantify feature impact:

$$P(defective|x) = \sigma(\beta_0 + \beta^\top x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}} \tag{1}$$

Decision Trees

Decision Trees sort data through binary splits based on sensor thresholds. We use Gini Impurity, as it is numerically stable under extreme class imbalance:

$$Gini(S) = \sum_{k \in \{0,1\}} p_k(1 - p_k) \tag{2}$$

Splits are evaluated by their reduction in impurity:

$$\Delta = Gini(S) - \left(\frac{|S_L|}{|S|} Gini(S_L) + \frac{|S_R|}{|S|} Gini(S_R) \right) \tag{3}$$

Based on 5-Fold Cross Validation, the optimal maximum depth was set to 5 to prevent overfitting.

Random Forest

Random Forest aggregates multiple trees trained on bootstrap samples and random feature subsets, yielding a majority vote prediction:

$$\hat{y} = \text{mode}\{\hat{y}^{(1)}, \hat{y}^{(2)}, \ldots, \hat{y}^{(N)}\} \tag{4}$$

We evaluate both Balanced Random Forest and Cost-Sensitive Random Forest to address the severe class imbalance in the dataset. Balanced Random Forest uses internal undersampling for each tree, meaning each bootstrap sample is adjusted so that the defective class is better represented during training. Cost-Sensitive Random Forest keeps the original training distribution but applies class-weighted impurity, giving higher penalty to mistakes on defective samples. Comparing both approaches allows us to test whether defect detection improves more from changing the sample distribution or from changing the cost of minority-class errors.

Boosting (AdaBoost, XGBoost, LightGBM, CatBoost)

Boosting builds an ensemble by repeatedly reweighting samples to focus on previous mistakes. For our baseline, AdaBoost, the error is:

$$\epsilon_i = \sum_j w_j^i \delta(C_i(x_j) \neq y_j) \tag{5}$$

Classifier importance:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \tag{6}$$

Sample weights update:

$$w_j^{i+1} = \frac{w_j^i}{Z_i} \times \begin{cases} e^{-\alpha_i}, & C_i(x_j) = y_j \\ e^{\alpha_i}, & C_i(x_j) \neq y_j \end{cases} \tag{7}$$

We extend our analysis to advanced gradient boosting frameworks (XGBoost, LightGBM, and CatBoost) to gauge performance gains and processing efficiency against the AdaBoost baseline.

3.3 Data Split Comparison

Because the defective class represents only a very small portion of the dataset, model performance may be sensitive to how the data are divided into training and testing sets. A split with a larger training portion, such as 90/10, gives the model more examples to learn from, including more rare defective samples. However, it also leaves fewer defective samples in the test set, which can make metrics more sensitive to small changes in the number of false negatives. In contrast, a 70/30 split provides a larger test set and a more stable estimate of generalization performance, but it reduces the number of defective samples available during training.

To evaluate this trade-off, we compare three stratified train/test split ratios: 90/10, 80/20, and 70/30. Stratification is used so that each split preserves approximately the same defective/non-defective class proportion as the full dataset. This is especially important for this project because random splitting without stratification could produce a test set with too few defective examples, making recall, false negative rate, and the FN & FP Score unreliable.

The purpose of the split comparison is not only to identify the split with the best score, but also to determine whether model rankings remain stable across different evaluation conditions. If a model performs well only under one split ratio, its performance may be dependent on a favorable partition of rare defective samples. However, if a model remains competitive across multiple splits, this provides stronger evidence that it generalizes to unseen manufacturing data. For this reason, split sensitivity is treated as part of the model evaluation process rather than only a preprocessing decision.

3.4 Hyperparameter Tuning

Each model underwent hyperparameter tuning using 5-fold cross-validation, utilizing a customized penalty function that evaluated the trade-off between FN and FP weights. Table 1 summarizes the default and tuned hyperparameters across all models. Notably, the optimal hyperparameters remained consistent across all FN weight values (0.5–1.0), suggesting that the models' best configurations are robust regardless of how aggresive the scoring weight is penalizing rates of FN (FNR).

Table 1: Model Hyperparameters: Default vs. Tuned (Consistent Across All FN Weights)

Model	Parameter	Default	Tuned
Logistic Regression	C	1.0	0.1
	l1_ratio	0.5	1
Decision Tree	max_depth	5	5
	min_leaf	1	1
Balanced RF	max_depth	None	10
	n_estimators	100	100
Cost-Sensitive RF	max_depth	None	5
	n_estimators	100	300
XGBoost	max_depth	6	3
	learning_rate	0.3	0.05
	n_estimators	100	300
LightGBM	max_depth	-1	3
	learning_rate	0.1	0.05
	n_estimators	100	100
CatBoost	max_depth	6	6
	learning_rate	0.03	0.05
	iterations	1000	300

4 Experimental Design

4.1 Model Selection Criteria

We evaluate simple models alongside complex ensembles to consider the trade-off between interpretability and predictive performance. Our experimental phase primarily evaluates 120 unique scenarios via our custom "FN & FP Score" System, as well as secondary evaluation with other relevant metrics in the case that predictive ability (according to our custom scoring system) and interpretability do not significantly emphasize a best choice model. Our experimental phase evaluates 720 unique scenarios (incorporating 8 models, 5 sampling techniques, 3 train/test splits, and 6 custom weight profiles) via our custom "FN & FP Score" System. We also conduct secondary evaluations with other relevant metrics (such as PR-AUC and F1-minority score) in the event that predictive ability and interpretability do not significantly emphasize a clear best-choice model.

Conceptual model transparency and interpretability are also heavily considered for final model suggestion/recommendation.

4.2 "FN & FP Score" System for Evaluating Model Prediction Ability

In many classification tasks, reducing false negatives (FN) is often more critical than reducing false positives (FP). To systematically evaluate models with this consideration, we propose a weighted scoring system that incorporates both the false negative rate (FNR) and false positive rate (FPR).

4.2.1 Weighted Score Definition

We define the **"FN & FP Score"** for a model as a weighted sum of the FN Rate and FP Rate:

$$\text{FN \& FP Score} = (\text{FN weighting}) \times (\text{FN Rate}) + (\text{FP weighting}) \times (\text{FP Rate}) \tag{8}$$

where FN weighting and FP weighting are non-negative weights reflecting the relative importance of reducing false negatives versus false positives.

4.2.2 Auxiliary Equation

Assuming the weights are complementary, we can express the FP weighting in terms of the FN weighting:

$$\text{FP weighting} = 1 - \text{FN weighting} \tag{9}$$

4.2.3 Equivalent Equation

Substituting the complementary weighting into the score formula yields:

$$\text{FN \& FP Score} = (\text{FN weighting}) \times (\text{FN Rate}) + (1 - \text{FN weighting}) \times (\text{FP Rate}) \tag{10}$$

This equivalent equation shows that the score is a function of three components: FN weighting, FNR, and FPR.

4.2.4 Model Ranking Using the "FN & FP Score"

The "FN & FP" Score allows ranking models according to their performance while prioritizing reduction of false negatives. For a given FN weighting, the **lowest score** indicates the **best model** at that weighting. By varying FN weighting (e.g., 50%, 60%, ..., 100%), we can evaluate model performance under different prioritization schemes.

This approach provides a flexible and interpretable framework for model selection, emphasizing the trade-off between FN and FP according to domain-specific or safetyrequirements.

4.2.5 Subjectivity of FN Weighting

The FN weighting metric is subject to change based on organizational priorities. For example, incorporating the severe ethical or safety consequences of missing a defect could heavily increase the FN weighting. A completely unbiased, cost-based method for calculating the FN weighting is as follows:

$$\text{FN weighting} = \frac{\text{Cost of FN}}{\text{Cost of FN} + \text{Cost of FP}} \tag{11}$$

In this case, the respective costs could represent labor, material waste, system downtime, or reputational damage to the company.

4.3 Other Relevant Metrics

Due to the extreme class imbalance (99.8% non-defective to 0.2% defective), standard accuracy and ROC AUC can be highly misleading [5]. Instead, recall, precision, Area Under the Precision-Recall Curve (PR-AUC), and the F1-minority score are used as secondary metrics to evaluate our models. Summary visualizations of these metrics are embedded within our attached Jupyter Notebook. These metrics serve as a vital secondary evaluative criteria to ensure the models aren't achieving high recall simply by blindly flagging every item as a defect.

5 Results

5.1 Comparing Predictive Ability

Bar charts displaying the top 10 scoring models per FN weighting are provided in Appendix B. In this context, a lower score indicates superior predictive ability for defect detection at a specific weight.

The results of our study demonstrate that a class-weighted logistic regression model with an 80/20 split outperforms all other tested models at 50% and 60% FN weightings, and ranks second best across all other weightings. For the 70%, 80%, 90%, and 100% FN weightings, a random undersampling AdaBoost model with a 90/10 split achieves the best predictive performance.

5.2 Comparing Interpretability

All models other than logistic regression that ranked within the top 10 for each weighting category utilize some form of boosting. While these ensemble techniques demonstrate strong predictive

performance, the logistic regression model provides superior interpretability compared to its boosting-based counterparts. This high level of transparency is particularly advantageous for defect detection, as it allows stakeholders to easily interpret the influence of individual features on the model's output.

This clarity comes from the fact that logistic regression represents predictions as a weighted combination of input features, where each factor has a direct and interpretable contribution. In contrast, boosting models combine many decision rules across multiple trees, creating more complex interactions that are harder to interpret at a glance. While these models are often more powerful, their structure can obscure the overall reasoning process. By remaining relatively simple, logistic regression allows experts to more easily understand which factors influence defects, without relying heavily on additional interpretation tools.

5.3 Secondary Metric Comparison

Secondary metrics were reviewed to confirm whether the top-ranked models under the FN & FP Score also performed well under standard imbalanced-classification measures. Recall, precision, F1-minority, PR-AUC, and ROC-AUC comparisons are shown in Appendix B, Figures 2–5.

As shown in Figure 5, AdaBoost with random undersampling achieved the highest peak recall at 0.9545. Class-weighted Logistic Regression followed closely at 0.9432, while LightGBM, XGBoost, and CatBoost each reached 0.9318 under their best configurations. This means the recommended Logistic Regression model gave up only about 0.0113 recall compared with the strongest recall model, while maintaining much stronger interpretability. The recall heatmap in Figure 4 also shows that random undersampling produced the strongest recall for several ensemble models, including AdaBoost, CatBoost, XGBoost, and LightGBM. However, Logistic Regression performed best with class weighting, suggesting that adjusting the learning penalty was more effective for the linear model than changing the sample distribution.

The recall-versus-precision plot in Figure 2 shows why recall alone was not enough to select the final model. Several high-recall configurations had noticeably lower precision, meaning they detected many defective samples but also produced more false alarms. The strongest 80/20 class-weighted Logistic Regression configuration achieved recall of 0.9432, precision of 0.0647, F1-minority of 0.1212, PR-AUC of 0.8019, and ROC-AUC of 0.9880. A similar 90/10 class-weighted Logistic Regression configuration had slightly lower recall at 0.9318, but higher precision at 0.0858 and higher F1-minority at 0.1571. Overall, the secondary metrics support the final recommendation because Logistic Regression remained competitive across standard minority-class metrics while remaining easier to interpret than the strongest boosting models.

6 Conclusion and Future Work

6.1 Our Recommended Model

Our recommended model across all FN weights is Class-weighted Logistic Regression with a 80/20 split.

- **Performance:** Consistently achieves top-tier results, ranking first for 50% and 60% weights and at least second across all other categories.
- **Interpretability:** Provides greater transparency and feature insight than AdaBoost, which is the only model to exceed its performance at weights $\geq 70\%$.

6.2 Limitations of Our Study

This study has several methodological and practical limitations. We benchmarked our models across 720 scenarios to ensure robust validation. However, we encountered a structural limitation regarding the distinction between *cost-sensitive learning* and *cost-sensitive evaluation*. Because our custom weights were applied via `make_scorer` in `GridSearchCV`, the formula acted purely as an external evaluator rather than being passed directly into the `.fit()` function. Consequently, the models used a default 50% decision threshold on balanced data, resulting in hypersensitive predictions that flagged a large portion of the dataset to avoid false negatives. Despite this, our custom formula successfully standardized the selection process post-prediction, providing a robust platform to equally compare

how well the algorithms handled imbalance while maintaining interpretability. Beyond the modeling architecture, the dataset itself presents challenges. It is extremely imbalanced (only 458 defects out of nearly 230,000 records) and consists of numerical features lacking contextual descriptions, limiting generalization and requiring domain knowledge for practical interpretation. Finally, the static nature of the dataset does not account for real-time production variability.

6.3 Potential Future Improvements

Future work should transition from post-training evaluation to true cost-sensitive optimization. Instead of retraining models for every weighting scenario, the pipeline should either dynamically inject weights directly into the algorithms during training (e.g., via `class_weight`), or utilize post-training threshold tuning (sliding the defect trigger below 50%) to optimize the decision boundary without redundant computation. Additionally, incorporating more defective sample data across different operating conditions will help the models better learn minority-class patterns. Finally, deploying the selected models in a real-time, streaming environment is necessary to evaluate their performance against production drift and determine periodic retraining intervals.

References

[1] E. G. Zahrani, F. Hojati, A. Daneshi, B. Azarhoushang, and J. Wilde. Application of machine learning to predict the product quality and geometry in circular laser grooving process. *Procedia CIRP*, 94:474–480, 2020.

[2] M. Geng, H. Ma, S. Liu, Z. Zhou, L. Xing, Y. Ai, and W. Zhang. A data-driven approach for internal crack prediction in continuous casting of HSLA steels using CTGAN and CatBoost. *Materials*, 18:3599, 2025.

[3] J. Kafunah, M. I. Ali, and J. G. Breslin. Handling imbalanced datasets for robust deep neural network-based fault detection in manufacturing systems. *Applied Sciences*, 11:9783, 2021.

[4] B. Liu, F. Gao, and Y. Li. Cost-sensitive YOLOv5 for detecting surface defects of industrial products. *Sensors*, 23:2610, 2023.

[5] M. Imani, M. Joudaki, A. Bagheri, and H. R. Arabnia. Why ROC-AUC is misleading for highly imbalanced data: In-depth evaluation of MCC, F2-Score, H-Measure, and AUC-based metrics across diverse classifiers. *Technologies*, 14(1):54, 2026.

[6] ScienceDirect. Minority class. *ScienceDirect Topics*. Retrieved March 5, 2025.

Appendix

A. Detailed Scoring Results

Table 2: "FN & FP Scores" and Error Rates for Various Scenarios

FN Weight	Rank	Scenario	"FN & FP Score"	FNR	FPR
0.5	1	80/20 | Class-Weighted | Logistic Regression	0.041515208	0.056818182	0.026212234
0.5	2	90/10 | Class-Weighted | Logistic Regression	0.043644492	0.068181818	0.019107166
0.5	3	90/10 | Random Oversampling | Logistic Regression	0.044191036	0.068181818	0.020200254
0.5	4	90/10 | Random Undersampling | CatBoost	0.045174814	0.068181818	0.022167811
0.5	5	90/10 | SMOTE | Logistic Regression	0.045459017	0.068181818	0.022736216
0.5	6	80/20 | SMOTE | Logistic Regression	0.04549181	0.068181818	0.022801801
0.5	7	90/10 | Random Undersampling | AdaBoost	0.046884502	0.045454545	0.048314459
0.5	8	80/20 | Random Undersampling | CatBoost	0.047295404	0.068181818	0.02640899
0.5	9	90/10 | Random Undersampling | XGBoost	0.047645192	0.068181818	0.027108565
0.5	10	90/10 | Random Undersampling | LightGBM	0.047776362	0.068181818	0.027370906
0.6	1	80/20 | Class-Weighted | Logistic Regression	0.044575803	0.056818182	0.026212234
0.6	2	90/10 | Random Undersampling | AdaBoost	0.046598511	0.045454545	0.048314459
0.6	3	90/10 | Class-Weighted | Logistic Regression	0.048551957	0.068181818	0.019107166
0.6	4	90/10 | Random Oversampling | Logistic Regression	0.048989192	0.068181818	0.020200254
0.6	5	90/10 | Random Undersampling | CatBoost	0.049776215	0.068181818	0.022167811
0.6	6	90/10 | SMOTE | Logistic Regression	0.049995539	0.068181818	0.022736216
0.6	7	80/20 | SMOTE | Logistic Regression	0.050027798	0.068181818	0.022801801
0.6	8	80/20 | Random Undersampling | CatBoost	0.051463556	0.068181818	0.02640899
0.6	9	90/10 | Random Undersampling | XGBoost	0.051723215	0.068181818	0.027108565
0.6	10	90/10 | Random Undersampling | LightGBM	0.051854385	0.068181818	0.027370906
0.7	1	90/10 | Random Undersampling | AdaBoost	0.046312519	0.045454545	0.048314459
0.7	2	80/20 | Class-Weighted | Logistic Regression	0.047636398	0.056818182	0.026212234
0.7	3	90/10 | Class-Weighted | Logistic Regression	0.053459423	0.068181818	0.019107166
0.7	4	90/10 | Random Oversampling | Logistic Regression	0.053787349	0.068181818	0.020200254
0.7	5	90/10 | Random Undersampling | CatBoost	0.054377616	0.068181818	0.022167811

Continued on next page

FN Weight	Rank	Scenario	"FN & FP Score"	FNR	FPR
0.7	6	90/10 | SMOTE | Logistic Regression	0.054532061	0.068181818	0.022736216
0.7	7	80/20 | SMOTE | Logistic Regression	0.054563785	0.068181818	0.022801801
0.7	8	80/20 | Random Undersampling | CatBoost	0.055631707	0.068181818	0.02640899
0.7	9	90/10 | Random Undersampling | XGBoost	0.055801238	0.068181818	0.027108565
0.7	10	90/10 | Random Undersampling | LightGBM	0.055932408	0.068181818	0.027370906
0.8	1	90/10 | Random Undersampling | AdaBoost	0.046026528	0.045454545	0.048314459
0.8	2	80/20 | Class-Weighted | Logistic Regression	0.050696993	0.056818182	0.026212234
0.8	3	90/10 | Class-Weighted | Logistic Regression	0.058366889	0.068181818	0.019107166
0.8	4	90/10 | Random Oversampling | Logistic Regression	0.058585505	0.068181818	0.020200254
0.8	5	90/10 | Random Undersampling | CatBoost	0.058979016	0.068181818	0.022167811
0.8	6	90/10 | SMOTE | Logistic Regression	0.059068582	0.068181818	0.022736216
0.8	7	80/20 | SMOTE | Logistic Regression	0.059099773	0.068181818	0.022801801
0.8	8	80/20 | Random Undersampling | CatBoost	0.059799859	0.068181818	0.02640899
0.8	9	90/10 | Random Undersampling | XGBoost	0.059879261	0.068181818	0.027108565
0.8	10	90/10 | Random Undersampling | LightGBM	0.060010431	0.068181818	0.027370906
0.9	1	90/10 | Random Undersampling | AdaBoost	0.045740536	0.045454545	0.048314459
0.9	2	80/20 | Class-Weighted | Logistic Regression	0.053757588	0.056818182	0.026212234
0.9	3	90/10 | Class-Weighted | Logistic Regression	0.063274354	0.068181818	0.019107166
0.9	4	90/10 | Random Oversampling | Logistic Regression	0.063383661	0.068181818	0.020200254
0.9	5	90/10 | Random Undersampling | CatBoost	0.063580417	0.068181818	0.022167811
0.9	6	90/10 | SMOTE | Logistic Regression	0.063605104	0.068181818	0.022736216
0.9	7	80/20 | SMOTE | Logistic Regression	0.06363576	0.068181818	0.022801801
0.9	8	80/20 | Random Undersampling | CatBoost	0.06396801	0.068181818	0.02640899
0.9	9	90/10 | Random Undersampling | XGBoost	0.063957284	0.068181818	0.027108565
0.9	10	90/10 | Random Undersampling | LightGBM	0.064088454	0.068181818	0.027370906
1	1	90/10 | Random Undersampling | AdaBoost	0.045454545	0.045454545	0.048314459
1	2	80/20 | Class-Weighted | Logistic Regression	0.056818182	0.056818182	0.026212234
1	3	90/10 | Class-Weighted | Logistic Regression	0.068181818	0.068181818	0.019107166

Continued on next page

FN Weight	Rank	Scenario	"FN & FP Score"	FNR	FPR
1	4	90/10 | Random Oversampling | Logistic Regression	0.068181818	0.068181818	0.020200254
1	5	90/10 | Random Undersampling | CatBoost	0.068181818	0.068181818	0.022167811
1	6	90/10 | SMOTE | Logistic Regression	0.068181818	0.068181818	0.022736216
1	7	80/20 | SMOTE | Logistic Regression	0.068181818	0.068181818	0.022801801
1	8	80/20 | Random Undersampling | CatBoost	0.068181818	0.068181818	0.02640899
1	9	90/10 | Random Undersampling | XGBoost	0.068181818	0.068181818	0.027108565
1	10	90/10 | Random Undersampling | LightGBM	0.068181818	0.068181818	0.027370906

B. Bar Charts: Top 10 Models Per Weighting

The following bar charts include the top 10 scoring models for each False Negative weight (50%-100%), along with their "FN & FP Scores".

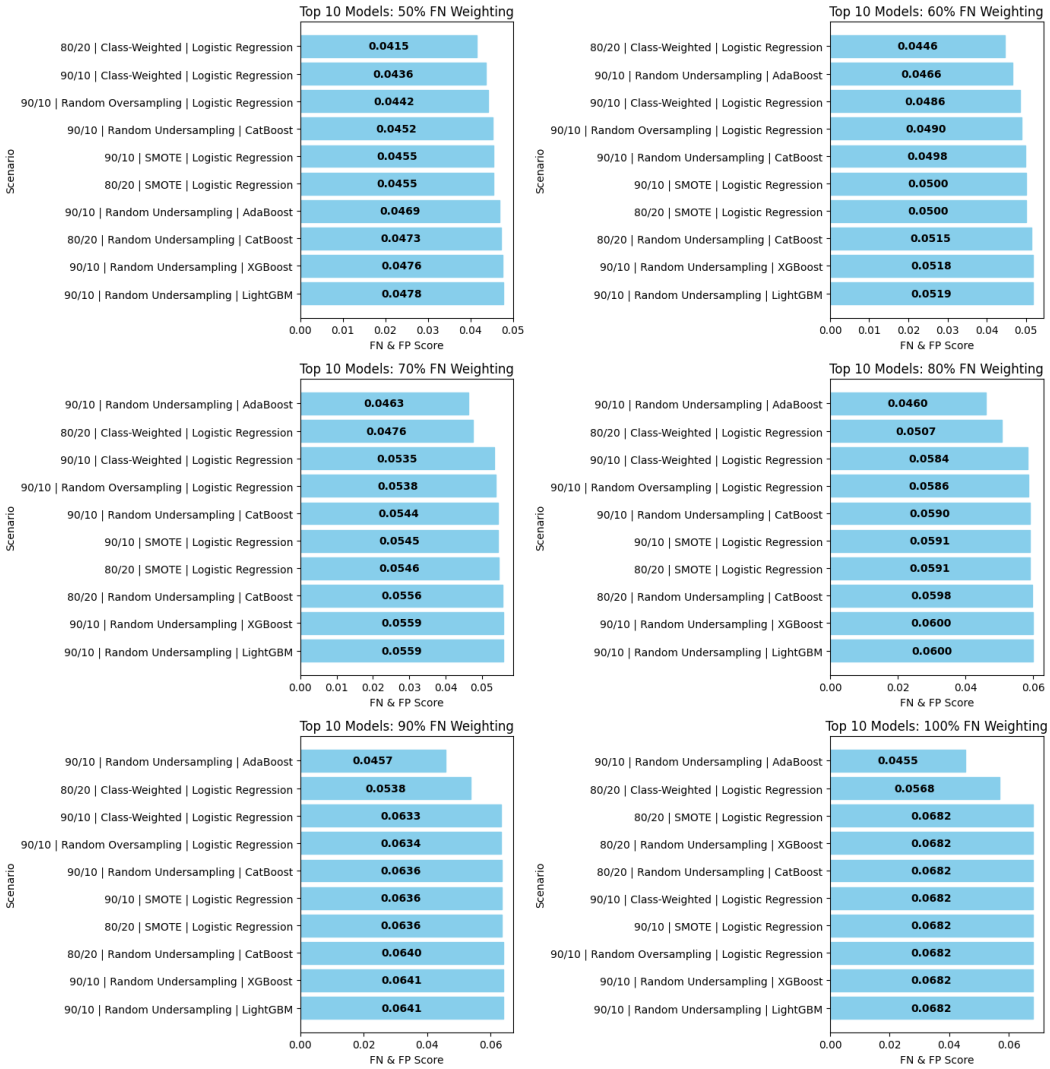

Figure 1: Model ranking results using the custom FN & FP Score.

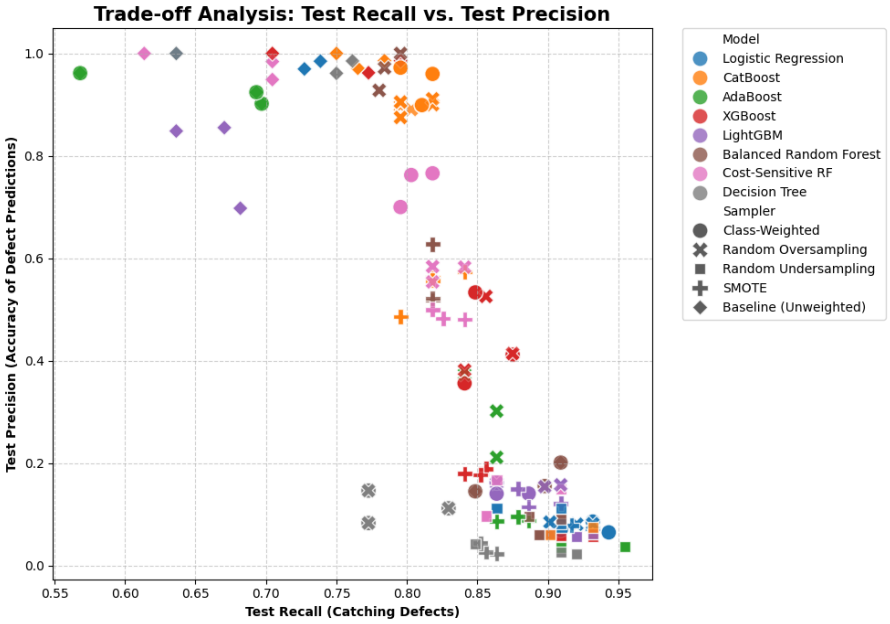

Figure 2: Recall and precision trade-off across model and sampling combinations. Higher recall indicates stronger defect detection, while lower precision indicates more false alarms.

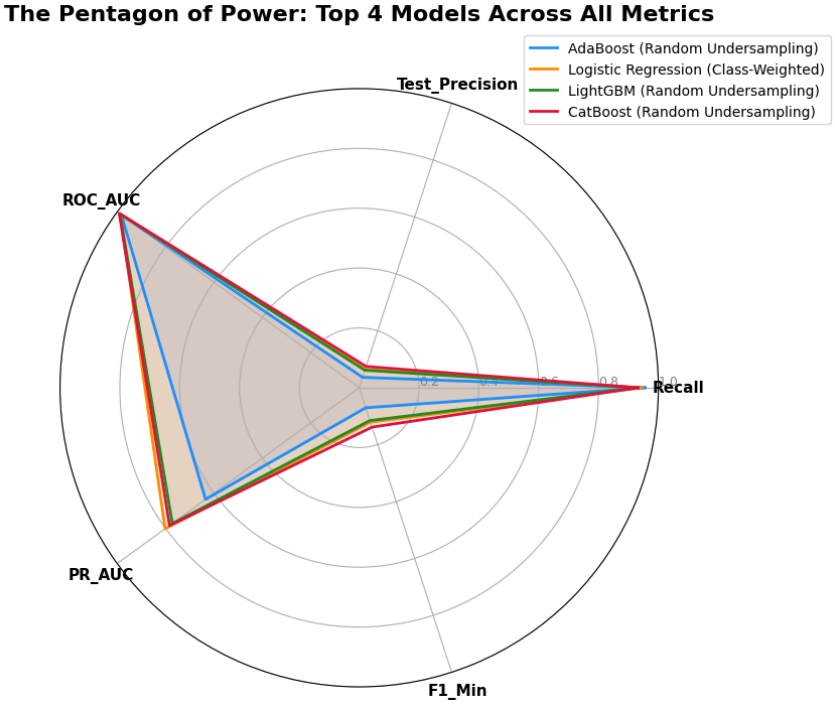

Figure 3: Multi-metric comparison of the top model configurations using recall, precision, F1-minority, PR-AUC, and ROC-AUC.

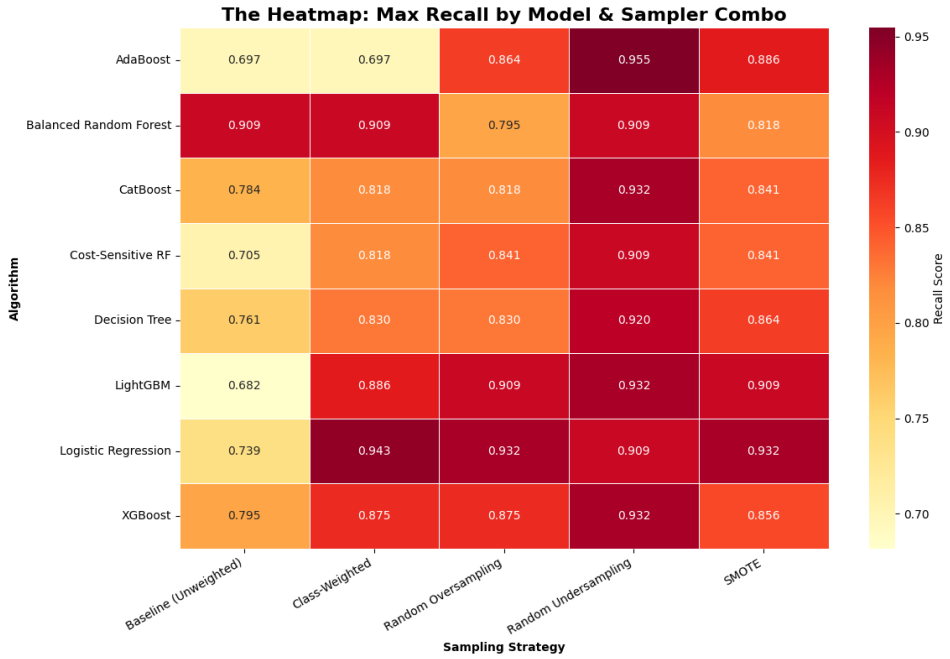

Figure 4: Recall heatmap by model type and sampling strategy. This figure highlights which model-sampler combinations were most effective at detecting defective samples.

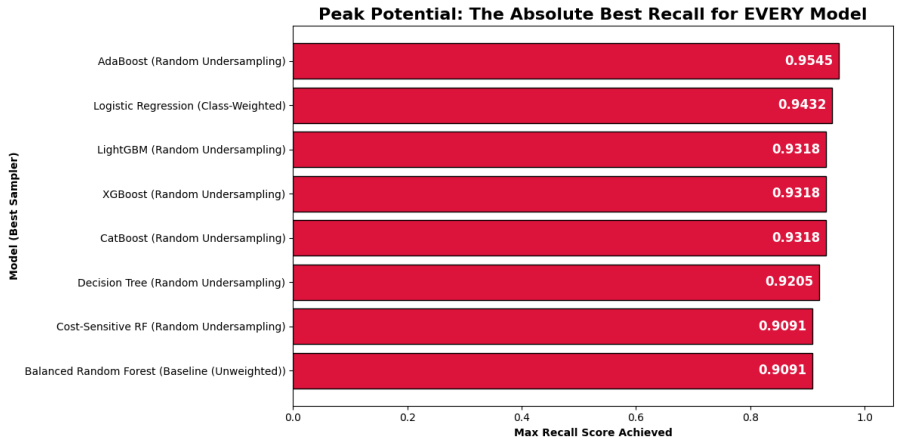

Figure 5: Peak recall achieved by each model type across tested sampling strategies and split ratios.